

SQL Window Functions Complete Notes

Contents

1	Introduction	3
2	Sample Table	3
3	PARTITION BY	3
4	ORDER BY	3
5	ROWS Frame	4
6	RANGE Frame	4
7	Ranking Window Functions	4
7.1	1. ROW_NUMBER()	4
7.2	2. RANK()	4
7.3	3. DENSE_RANK()	5
7.4	4. NTILE()	5
8	Aggregate Window Functions	5
8.1	5. SUM()	5
8.2	6. AVG()	5
8.3	7. MIN()	6
8.4	8. MAX()	6
8.5	9. COUNT()	6
9	Value Window Functions	6
9.1	10. LAG()	6
9.2	11. LEAD()	6
9.3	12. FIRST_VALUE()	7
9.4	13. LAST_VALUE()	7
9.5	14. NTH_VALUE()	7
10	Distribution Functions	7
10.1	15. CUME_DIST()	7
10.2	16. PERCENT_RANK()	8
11	Percentile Functions	8
11.1	17. PERCENTILE_CONT()	8
11.2	18. PERCENTILE_DISC()	8
12	Running Total	8

13 Moving Average	9
14 Highest Salary Department Wise	9
15 Second Highest Salary Department Wise	9
16 Difference Between Ranking Functions	9
17 Complete List of SQL Window Functions	10
18 Important Interview Questions	10
19 Conclusion	11

1 Introduction

Window Functions perform calculations across a set of rows that are related to the current row without collapsing the result into a single row.

Unlike GROUP BY, window functions return one output for every row.

General Syntax:

```
1 FUNCTION_NAME(expression)
2 OVER (
3     [PARTITION BY column]
4     [ORDER BY column]
5     [ROWS/RANGE frame]
6 )
```

2 Sample Table

Assume the following Employee table.

EmpID	Name	Department	Salary
101	Aftab	IT	50000
102	Rahul	IT	60000
103	Aman	HR	45000
104	Priya	HR	70000
105	Mohit	Sales	55000
106	Neha	Sales	65000

3 PARTITION BY

Splits rows into groups before applying the window function.

Example

```
1 SELECT *,
2 SUM(Salary) OVER(PARTITION BY Department)
3 AS Department_Total
4 FROM Employee;
```

4 ORDER BY

Defines the order within each partition.

```
1 SELECT *,
2 ROW_NUMBER() OVER(
3 ORDER BY Salary DESC
4 ) AS RowNum
5 FROM Employee;
```

5 ROWS Frame

Specifies the physical rows.

```
1 SUM(Salary)
2 OVER(
3 ORDER BY Salary
4 ROWS BETWEEN
5 2 PRECEDING
6 AND CURRENT ROW
7 )
```

6 RANGE Frame

Specifies logical ranges instead of row numbers.

```
1 SUM(Salary)
2 OVER(
3 ORDER BY Salary
4 RANGE BETWEEN
5 UNBOUNDED PRECEDING
6 AND CURRENT ROW
7 )
```

7 Ranking Window Functions

7.1 1. ROW_NUMBER()

Assigns unique numbers.

```
1 SELECT
2 Name,
3 Salary,
4 ROW_NUMBER() OVER(
5 ORDER BY Salary DESC
6 ) AS RowNo
7 FROM Employee;
```

7.2 2. RANK()

Assigns rank with gaps.

```
1 SELECT
2 Name,
3 Salary,
4 RANK() OVER(
5 ORDER BY Salary DESC
6 ) AS RankNo
7 FROM Employee;
```

Example

90 -> Rank 1

90 -> Rank 1

80 -> Rank 3

7.3 3. DENSE_RANK()

Assigns rank without gaps.

```
1 SELECT
2 Name,
3 Salary,
4 DENSE_RANK() OVER(
5 ORDER BY Salary DESC
6 ) AS DenseRank
7 FROM Employee;
```

Example

90 -> Rank 1

90 -> Rank 1

80 -> Rank 2

7.4 4. NTILE()

Divides rows into equal groups.

```
1 SELECT
2 Name,
3 Salary,
4 NTILE(4)
5 OVER(ORDER BY Salary)
6 AS Quartile
7 FROM Employee;
```

8 Aggregate Window Functions

8.1 5. SUM()

```
1 SELECT
2 Name,
3 Salary,
4 SUM(Salary)
5 OVER()
6 AS TotalSalary
7 FROM Employee;
```

8.2 6. AVG()

```
1 SELECT
2 Name,
3 Salary,
4 AVG(Salary)
5 OVER()
6 AS AvgSalary
7 FROM Employee;
```

8.3 7. MIN()

```
1 SELECT
2 Name,
3 Salary,
4 MIN(Salary)
5 OVER()
6 AS MinSalary
7 FROM Employee;
```

8.4 8. MAX()

```
1 SELECT
2 Name,
3 Salary,
4 MAX(Salary)
5 OVER()
6 AS MaxSalary
7 FROM Employee;
```

8.5 9. COUNT()

```
1 SELECT
2 Name,
3 COUNT(*)
4 OVER()
5 AS TotalEmployees
6 FROM Employee;
```

9 Value Window Functions

9.1 10. LAG()

Returns previous row value.

```
1 SELECT
2 Name,
3 Salary,
4 LAG(Salary)
5 OVER(ORDER BY Salary)
6 AS PreviousSalary
7 FROM Employee;
```

9.2 11. LEAD()

Returns next row value.

```
1 SELECT
2 Name,
3 Salary,
4 LEAD(Salary)
5 OVER(ORDER BY Salary)
6 AS NextSalary
7 FROM Employee;
```

9.3 12. FIRST_VALUE()

Returns first value.

```
1 SELECT
2 Name,
3 Salary,
4 FIRST_VALUE(Salary)
5 OVER(
6 ORDER BY Salary DESC
7 )
8 AS HighestSalary
9 FROM Employee;
```

9.4 13. LAST_VALUE()

Returns last value.

```
1 SELECT
2 Name,
3 Salary,
4 LAST_VALUE(Salary)
5 OVER(
6 ORDER BY Salary
7 ROWS BETWEEN
8 UNBOUNDED PRECEDING
9 AND UNBOUNDED FOLLOWING
10 )
11 AS LowestSalary
12 FROM Employee;
```

9.5 14. NTH_VALUE()

Returns nth row.

```
1 SELECT
2 Name,
3 Salary,
4 NTH_VALUE(Salary,3)
5 OVER(
6 ORDER BY Salary DESC
7 ROWS BETWEEN
8 UNBOUNDED PRECEDING
9 AND UNBOUNDED FOLLOWING
10 )
11 AS ThirdHighest
12 FROM Employee;
```

10 Distribution Functions

10.1 15. CUME_DIST()

Returns cumulative distribution.

```
1 SELECT
2 Name,
3 Salary,
```

```

4 CUME_DIST()
5 OVER(
6 ORDER BY Salary
7 )
8 AS CumDist
9 FROM Employee;

```

10.2 16. PERCENT_RANK()

Returns relative rank.

```

1 SELECT
2 Name,
3 Salary,
4 PERCENT_RANK()
5 OVER(
6 ORDER BY Salary
7 )
8 AS PercentRank
9 FROM Employee;

```

11 Percentile Functions

11.1 17. PERCENTILE_CONT()

Continuous percentile.

```

1 SELECT
2 PERCENTILE_CONT(0.5)
3 WITHIN GROUP
4 (ORDER BY Salary)
5 OVER()
6 AS MedianSalary;

```

11.2 18. PERCENTILE_DISC()

Discrete percentile.

```

1 SELECT
2 PERCENTILE_DISC(0.5)
3 WITHIN GROUP
4 (ORDER BY Salary)
5 OVER()
6 AS MedianSalary;

```

12 Running Total

```

1 SELECT
2 Name,
3 Salary,
4 SUM(Salary)
5 OVER(
6 ORDER BY EmpID
7 ) AS RunningTotal
8 FROM Employee;

```

13 Moving Average

```
1 SELECT
2 Name ,
3 Salary ,
4 AVG(Salary)
5 OVER(
6 ORDER BY EmpID
7 ROWS BETWEEN
8 2 PRECEDING
9 AND CURRENT ROW
10 )
11 AS MovingAverage
12 FROM Employee;
```

14 Highest Salary Department Wise

```
1 SELECT
2 Name ,
3 Department ,
4 Salary ,
5 RANK()
6 OVER(
7 PARTITION BY Department
8 ORDER BY Salary DESC
9 )
10 AS RankNo
11 FROM Employee;
```

15 Second Highest Salary Department Wise

```
1 SELECT *
2 FROM
3 (
4 SELECT *,
5 DENSE_RANK()
6 OVER(
7 PARTITION BY Department
8 ORDER BY Salary DESC
9 )
10 AS RankNo
11 FROM Employee
12 ) X
13 WHERE RankNo=2;
```

16 Difference Between Ranking Functions

Function	Duplicate Rank	Gap
ROW_NUMBER()	No	No
RANK()	Yes	Yes
DENSE_RANK()	Yes	No
NTILE()	Groups	No

17 Complete List of SQL Window Functions

1. ROW_NUMBER()
2. RANK()
3. DENSE_RANK()
4. NTILE()
5. SUM()
6. AVG()
7. MIN()
8. MAX()
9. COUNT()
10. LAG()
11. LEAD()
12. FIRST_VALUE()
13. LAST_VALUE()
14. NTH_VALUE()
15. CUME_DIST()
16. PERCENT_RANK()
17. PERCENTILE_CONT()
18. PERCENTILE_DISC()

18 Important Interview Questions

1. What are Window Functions?
2. Difference between GROUP BY and Window Functions?
3. What is OVER()?
4. Difference between PARTITION BY and GROUP BY?
5. Difference between ROW_NUMBER(), RANK() and DENSE_RANK()?
6. What is NTILE()?
7. Explain LAG() and LEAD().
8. Explain FIRST_VALUE() and LAST_VALUE().
9. What is Running Total?
10. What is Moving Average?
11. Difference between ROWS and RANGE?

12. How to find second highest salary department wise?
13. How to calculate cumulative sum?
14. Explain PERCENT_RANK().
15. Explain CUME_DIST().

19 Conclusion

Window Functions are among the most powerful SQL features. They allow ranking, cumulative calculations, comparisons between rows, moving averages, and advanced analytical reporting while preserving all rows in the result set.

Mastering Window Functions is essential for Data Analysts, Data Scientists, SQL Developers, and Business Intelligence professionals.